

UNIVERSIDAD DEL VALLE
FACULTAD DE CIENCIAS EMPRESARIALES E INGENIERÍA



TRABAJO FINAL
Predicción de Cancelaciones de Reservas Hoteleras
Clasificación con Redes Neuronales Artificiales

ING. EN CIENCIA DE DATOS Y NEGOCIOS INTELIGENTES

MATERIA: BIG DATA

ESTUDIANTE:
AMIRA JIMENA ROMAN JUCHANI

Dataset: Hotel Booking Demand

Fuente: Kaggle

COCHABAMBA – BOLIVIA
2025

BIG DATA

TRABAJO FINAL

Recibido el:
10/ 06 /26Entregado el:
16 / 06 /26

Predicción de Cancelaciones de Reservas Hoteleras mediante Redes Neuronales Artificiales

Resumen

Este trabajo implementa y evalúa una Red Neuronal Artificial (RNA) de tipo Perceptrón Multicapa (MLP) para predecir si una reserva hotelera será cancelada, utilizando el dataset *Hotel Booking Demand* con 119,390 registros y 32 variables. Se aplicó un preprocesamiento exhaustivo que incluyó limpieza de datos, ingeniería de variables, codificación y normalización. La arquitectura MLP (128→64→32→1 neuronas) entrenada con el optimizador Adam y regularización mediante Dropout y EarlyStopping alcanzó una exactitud del 86.34% y un AUC-ROC de 0.9419 en el conjunto de prueba. Como modelo comparativo se utilizó Random Forest (200 árboles), obteniendo métricas similares (Accuracy: 86.57%, AUC: 0.9424). Ambos modelos demuestran alta capacidad predictiva, con el Random Forest marginalmente superior en métricas globales, aunque la RNA ofrece mayor capacidad de generalización en patrones complejos.

Palabras clave: Redes Neuronales Artificiales, Perceptrón Multicapa, Hotel Booking, Clasificación Binaria, Random Forest, Cancelaciones, Deep Learning, TensorFlow.

1 Introducción

1.1 Problema a Resolver

La industria hotelera enfrenta uno de sus mayores desafíos operativos en la gestión de cancelaciones de reservas. Una cancelación imprevista implica pérdida de ingresos, ineficiencia en la asignación de recursos (habitaciones, personal, insumos) y dificultad para gestionar la disponibilidad en tiempo real.

1.2 Relevancia del Problema

La capacidad de predecir con antelación si una reserva será cancelada permite a los hoteles:

- Optimizar la estrategia de sobreventa (*overbooking*) de manera controlada.
- Personalizar políticas de cancelación según el perfil del cliente.
- Maximizar la tasa de ocupación real y los ingresos por habitación.
- Mejorar la planificación de recursos humanos y de servicios.

1.3 Objetivo General

Desarrollar e implementar un modelo de clasificación basado en Redes Neuronales Artificiales (MLP) capaz de predecir con alta precisión si una reserva hotelera será cancelada, y compararlo con un modelo Random Forest para evaluar ventajas y limitaciones de ambos enfoques.

2 Descripción del Dataset

2.1 Información General

Table 1: Ficha técnica del dataset Hotel Booking Demand.

Característica	Detalle
Nombre	Hotel Booking Demand Dataset
Fuente original	Antonio, Almeida & Nunes (2019) – <i>Data in Brief</i> , Vol. 22
Plataforma	Kaggle (datasets/jessemostipak/hotel-booking-demand)
Filas (registros)	119,390 reservas
Columnas (variables)	32 variables originales
Período cubierto	Julio 2015 – Agosto 2017
Tipos de hotel	Resort Hotel y City Hotel
Variable objetivo	<code>is_canceled</code> (0 = No cancelada, 1 = Cancelada)

2.2 Variables del Dataset

El dataset contiene **19 variables numéricas** y **13 variables categóricas**. Las más relevantes para el modelo se describen a continuación:

Table 2: Variables clave del dataset y su descripción.

Variable	Tipo	Descripción
<code>is_canceled</code>	Binaria	Variable objetivo: 1=cancelada, 0=no cancelada
<code>lead_time</code>	Numérica	Días entre reserva y llegada
<code>hotel</code>	Categórica	Tipo de hotel (Resort / City)
<code>adr</code>	Numérica	Tarifa diaria promedio (Average Daily Rate)
<code>deposit_type</code>	Categórica	Tipo de depósito (No Deposit / Non Refund / Refundable)
<code>market_segment</code>	Categórica	Segmento de mercado (Direct, OTA, Corporate, etc.)
<code>total_of_special_reques</code>	Numérica	Número de peticiones especiales del huésped
<code>previous_cancellations</code>	Numérica	Número de cancelaciones previas del cliente
<code>booking_changes</code>	Numérica	Modificaciones realizadas en la reserva
<code>customer_type</code>	Categórica	Tipo de cliente (Transient, Contract, Group, etc.)
<code>stays_in_week_nights</code>	Numérica	Noches entre semana de la estancia
<code>stays_in_weekend_nights</code>	Numérica	Noches de fin de semana de la estancia

2.3 Distribución de la Variable Objetivo

Table 3: Distribución de clases en la variable objetivo `is_canceled`.

Clase	Registros	Porcentaje
0 – No Cancelada	75,166	62.97%
1 – Cancelada	44,224	37.03%
Total	119,390	100%

El dataset presenta un **desbalance moderado de clases** (ratio aproximado 63:37), lo que se consideró durante el diseño del modelo y la interpretación de métricas.

3 Análisis Exploratorio de Datos (EDA)

3.1 Estadísticas Descriptivas

El análisis de las variables numéricas reveló las siguientes estadísticas clave:

Table 4: Estadísticas descriptivas de variables numéricas seleccionadas.

Variable	Media	Std	Min	Max
lead_time	104.01	106.86	0	737
adr	101.83	50.54	0	5400
stays_in_week_nights	2.50	1.91	0	50
stays_in_weekend_nights	0.93	0.99	0	19
total_of_special_requests	0.57	0.79	0	5
booking_changes	0.22	0.65	0	21
previous_cancellations	0.09	0.84	0	26

3.2 Análisis de Valores Nulos

Se identificaron cuatro columnas con valores nulos antes del preprocesamiento:

Table 5: Columnas con valores nulos y estrategia de tratamiento.

Columna	Nulos	Porcentaje	Tratamiento
children	4	0.00%	Imputación con mediana
country	488	0.41%	Imputación con 'Unknown'
agent	16,340	13.69%	Eliminada (identificador)
company	112,593	94.31%	Eliminada (94% nulos)

3.3 Correlaciones con la Variable Objetivo

El análisis de correlación de Pearson reveló las variables con mayor relación lineal con `is_canceled`:

Table 6: Top correlaciones con la variable objetivo `is_canceled`.

Variable	Correlación (abs.)	Dirección
<code>lead_time</code>	0.2931	Positiva
<code>total_of_special_requests</code>	0.2347	Negativa
<code>required_car_parking_spaces</code>	0.1955	Negativa
<code>booking_changes</code>	0.1444	Negativa
<code>previous_cancellations</code>	0.1101	Positiva
<code>is_repeated_guest</code>	0.0848	Negativa

Interpretación clave: El `lead_time` es la variable con mayor correlación con las cancelaciones. Cuanto más tiempo de anticipación tiene una reserva, mayor es la probabilidad de cancelación, lo cual es coherente con la intuición del negocio hotelero: reservas hechas con mucha antelación tienden a ser canceladas con más frecuencia.

3.4 Análisis de Variables Categóricas

El análisis de las variables categóricas más relevantes mostró diferencias importantes en la tasa de cancelación:

- **deposit_type:** Las reservas con depósito *Non Refund* presentan tasas de cancelación extremadamente altas (cercasas al 99%), mientras que las sin depósito tienen tasas moderadas. Esto puede parecer contraintuitivo pero refleja que los sistemas de reservas clasifican como “canceladas” a reservas con restricciones muy estrictas.
- **market_segment:** Las reservas por canales de agencias de viaje online (OTA) presentan las mayores tasas de cancelación, mientras que los grupos corporativos son más estables.
- **customer_type:** Los clientes de tipo *Transient-Party* tienen mayores tasas de cancelación que los clientes de contrato o grupo.

4 Preprocesamiento

4.1 Estrategia General

Table 7: Estrategia de preprocesamiento aplicada.

Paso	Tipo	Descripción	Técnica
1	Limpieza	Eliminar columnas con fuga de datos o irrelevantes	Drop de <code>reservation_status</code> , <code>reservation_status_date</code> , <code>agent</code> , <code>company</code>
2	Valores nulos	Imputar o eliminar	Mediana para numéricos, 'Unknown' para categóricos
3	Ingeniería	Crear nuevas variables	<code>total_nights</code> , <code>total_guests</code> , <code>arrival_month_num</code>
4	Encoding	Convertir categóricas a numéricas	Label Encoding + Frequency Encoding para <code>country</code>
5	División	Separar train/test	<code>train_test_split</code> 80%/20% estratificado
6	Escalado	Normalizar variables numéricas	StandardScaler (ajuste solo en train)

4.2 Limpieza y Eliminación de Fuga de Datos

Fuga de datos (*data leakage*): Las columnas `reservation_status` y `reservation_status_date` revelan directamente el resultado de la reserva, por lo que fueron eliminadas obligatoriamente para evitar un sobreentrenamiento artificialmente alto.

Tras la limpieza inicial, el dataset quedó reducido de 32 a 28 columnas.

4.3 Ingeniería de Variables

Se crearon tres variables derivadas para enriquecer el conjunto de features:

Listing 1: Ingeniería de variables derivadas.

```
# Total de noches de estancia
df_proc['total_nights'] = (df_proc['stays_in_weekend_nights']
                          + df_proc['stays_in_week_nights'])

# Total de huéspedes por reserva
df_proc['total_guests'] = (df_proc['adults']
                          + df_proc['children']
                          + df_proc['babies'])

# Mes de llegada como numero
month_map = {'January':1, 'February':2, ..., 'December':12}
df_proc['arrival_month_num'] = df_proc['arrival_date_month'].map(
    month_map)
```

4.4 Codificación de Variables Categóricas

Se aplicó **Label Encoding** a las variables categóricas de baja y media cardinalidad:

Table 8: Variables categóricas y su número de categorías tras encoding.

Variable	Nro. Categorías	Técnica
hotel	2	Label Encoding
meal	5	Label Encoding
market_segment	8	Label Encoding
distribution_channel	5	Label Encoding
reserved_room_type	10	Label Encoding
assigned_room_type	12	Label Encoding
deposit_type	3	Label Encoding
customer_type	4	Label Encoding
country	Alta cardinalidad	Frequency Encoding

4.5 División y Escalado

El dataset procesado se dividió en:

- **Train:** 95,512 muestras (80%) — usado para entrenamiento y validación interna.
- **Test:** 23,878 muestras (20%) — reservado exclusivamente para evaluación final.

El escalado con **StandardScaler** se ajustó *únicamente* sobre el conjunto de entrenamiento para evitar fuga de información estadística del test al train. Las dimensiones finales del conjunto de features fueron: **29 variables** tras todo el preprocesamiento.

5 Fundamentos Teóricos del Modelo

5.1 Definición y Arquitectura de una RNA

Una **Red Neuronal Artificial (RNA)** es un modelo computacional inspirado en la estructura del cerebro biológico (Haykin, 2009). Está compuesta por unidades de procesamiento denominadas **neuronas artificiales** organizadas en capas:

- **Capa de entrada (*Input Layer*):** Recibe las variables predictoras. Una neurona por feature.
- **Capas ocultas (*Hidden Layers*):** Aprenden representaciones intermedias no lineales.
- **Capa de salida (*Output Layer*):** Produce la predicción final.

5.2 Base Matemática

5.2.1 Operación de una neurona Cada neurona en la capa l calcula una combinación lineal ponderada de sus entradas más un sesgo (*bias*), y luego aplica una función de activación no lineal:

$$z^{(l)} = \sum_{i=1}^n w_i^{(l)} x_i + b^{(l)} = \mathbf{w}^{(l)\top} \mathbf{x} + b^{(l)} \quad (1)$$

$$a^{(l)} = f(z^{(l)}) \quad (2)$$

donde x_i es la entrada i -ésima, w_i es el peso sináptico, b es el sesgo y $f(\cdot)$ es la función de activación.

5.2.2 Funciones de activación utilizadas **ReLU (*Rectified Linear Unit*)** — capas ocultas:

$$\text{ReLU}(z) = \max(0, z) \quad (3)$$

Sigmoid — capa de salida (clasificación binaria):

$$\sigma(z) = \frac{1}{1 + e^{-z}} \in (0, 1) \quad (4)$$

5.3 Función de Pérdida (Objetivo)

Para clasificación binaria se emplea la **Entropía Cruzada Binaria (Binary Cross-Entropy)**:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (5)$$

donde $y_i \in \{0, 1\}$ es la etiqueta real y $\hat{y}_i \in (0, 1)$ es la probabilidad predicha por el modelo.

5.4 Proceso de Entrenamiento — Backpropagation

El algoritmo de **retropropagación del error** (Rumelhart, Hinton & Williams, 1986) calcula el gradiente de la función de pérdida respecto a cada peso usando la regla de la cadena:

$$\frac{\partial \mathcal{L}}{\partial w_j^{(l)}} = \frac{\partial \mathcal{L}}{\partial a^{(l)}} \cdot \frac{\partial a^{(l)}}{\partial z^{(l)}} \cdot \frac{\partial z^{(l)}}{\partial w_j^{(l)}} \quad (6)$$

La actualización de pesos se realiza con el **optimizador Adam** (Kingma & Ba, 2015), que combina estimaciones adaptativas del primer y segundo momento del gradiente:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (7)$$

donde η es la tasa de aprendizaje, $\hat{m}_t$ es la media del gradiente corregida por sesgo, $\hat{v}_t$ es la varianza del gradiente corregida por sesgo, y $\epsilon = 10^{-8}$ es un término de estabilidad numérica.

5.5 Regularización con Dropout

El **Dropout** (Srivastava et al., 2014) desactiva aleatoriamente neuronas durante el entrenamiento con probabilidad p :

$$\tilde{h} = h \cdot \text{Bernoulli}(1 - p) \quad (8)$$

Este mecanismo previene el **sobreajuste** (*overfitting*) al forzar al modelo a aprender representaciones más robustas y generalizables.

5.6 Hiperparámetros Clave y su Impacto

Table 9: Hiperparámetros del modelo y justificación de su elección.

Hiperparámetro	Valor elegido	Impacto
Neuronas por capa	128 / 64 / 32	Permite capturar representaciones de complejidad decreciente
Capas ocultas	3	Profundidad suficiente para patrones no lineales moderados
Learning rate	0.001	Balance entre velocidad de convergencia y estabilidad
Batch size	512	Velocidad computacional sin perder estabilidad del gradiente
Epochs máximos	50 (con EarlyStopping)	Límite superior; el modelo entrenó las 50 épocas
Dropout rate	0.3 (capas 1–2) / 0.15 (capa 3)	Regularización progresiva, mayor en capas profundas
Activación oculta	ReLU	Evita el problema del gradiente desvaneciente
Activación salida	Sigmoid	Produce probabilidad en $[0, 1]$ para clasificación binaria
Optimizador	Adam	Adapta la tasa de aprendizaje por parámetro; robusto

5.7 Ventajas, Limitaciones y Casos de Uso

Table 10: Ventajas y limitaciones de las Redes Neuronales MLP.

Ventajas	Limitaciones
Aprende relaciones no lineales complejas	Baja interpretabilidad (“caja negra”)
Escalable a grandes volúmenes de datos	Alta demanda computacional
Flexible en arquitectura	Sensible a elección de hiperparámetros
Generaliza bien con regularización	Requiere preprocesamiento riguroso
Soporta múltiples tipos de variables	Propenso a sobreajuste sin regularización

Casos de uso reales: Detección de fraude bancario, reconocimiento de voz e imagen, sistemas de recomendación (Netflix, Spotify), predicción de demanda en retail y hotelería.

6 Implementación

6.1 Librerías Utilizadas

Table 11: Librerías principales utilizadas en la implementación.

Librería	Versión	Uso
TensorFlow / Keras	2.20.0	Construcción y entrenamiento de la RNA
Scikit-learn	última estable	Preprocesamiento, métricas, Random Forest
Pandas	2.2.2	Manipulación del dataset
NumPy	2.0.2	Operaciones matriciales
Matplotlib / Seaborn	últimas estables	Visualizaciones EDA y resultados

6.2 Arquitectura del Modelo

La arquitectura implementada es un **MLP (Multilayer Perceptron)** con la estructura:

Input (29) → Dense(128, ReLU) → BN → Dropout(0.3) →
Dense(64, ReLU) → BN → Dropout(0.3) →
Dense(32, ReLU) → Dropout(0.15) → Dense(1, Sigmoid)

Listing 2: Arquitectura de la Red Neuronal MLP implementada.

```
def construir_modelo(input_dim, lr=0.001, dropout_rate=0.3):
    model = keras.Sequential([
        layers.Input(shape=(input_dim,)),
        # Capa oculta 1
        layers.Dense(128, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(dropout_rate),           # Dropout 30%
        # Capa oculta 2
        layers.Dense(64, activation='relu'),
        layers.BatchNormalization(),
        layers.Dropout(dropout_rate),           # Dropout 30%
        # Capa oculta 3
        layers.Dense(32, activation='relu'),
```

```

        layers.Dropout(dropout_rate / 2),      # Dropout 15%
        # Capa de salida
        layers.Dense(1, activation='sigmoid')
    ], name='RNA_HotelBooking')

    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss=BinaryCrossentropy(),
        metrics=['accuracy', AUC(), Precision(), Recall()]
    )
    return model

```

El modelo tiene un total de **14,977 parámetros entrenables** (14,593 entrenables + 384 no entrenables de BatchNorm).

6.3 Estrategia de Entrenamiento

Se emplearon dos *callbacks* para optimizar el entrenamiento:

- **EarlyStopping:** Monitor `val_loss`, paciencia de 8 épocas, restaura los mejores pesos.
- **ReduceLROnPlateau:** Reduce la tasa de aprendizaje a la mitad si `val_loss` no mejora en 4 épocas (mínimo 10^{-6}).

El modelo entrenó las 50 épocas completas, alcanzando:

- **Mejor `val_loss`:** 0.2891
- **Mejor `val_accuracy`:** 87.02%

7 Modelo Comparativo — Random Forest

7.1 Justificación de la Elección

Se eligió **Random Forest** como modelo comparativo por las siguientes razones:

- Es un modelo ensamble robusto con excelente desempeño en datos tabulares.
- Ofrece interpretabilidad a través de la importancia de variables (*feature importance*).
- No requiere escalado de variables, lo que permite una comparación más directa.
- Es ampliamente utilizado en la industria como modelo de referencia (*baseline*).

7.2 Configuración del Modelo

Listing 3: Configuración del modelo Random Forest comparativo.

```

modelo_rf = RandomForestClassifier(
    n_estimators=200,      # 200 arboles
    max_depth=15,         # Profundidad maxima
    min_samples_split=10,  # Minimo para dividir nodo
    min_samples_leaf=5,    # Minimo en hojas
    n_jobs=-1,             # Paralelismo total
    random_state=42,
    class_weight='balanced' # Compensa desbalance de clases
)
modelo_rf.fit(X_train, y_train)

```

7.3 Importancia de Variables (Random Forest)

El análisis de importancia de variables del Random Forest identificó las siguientes como más relevantes para la predicción de cancelaciones:

1. `lead_time` (tiempo de anticipación de la reserva)
2. `adr` (tarifa diaria promedio)

3. `total_of_special_requests` (peticiones especiales)
4. `arrival_date_week_number` (semana de llegada)
5. `country_freq` (frecuencia del país de origen)
6. `deposit_type` (tipo de depósito)
7. `previous_cancellations` (cancelaciones previas)

Estos resultados son coherentes con las correlaciones encontradas en el EDA y con la intuición del negocio hotelero.

8 Resultados

8.1 Métricas en el Conjunto de Prueba

Table 12: Comparación de métricas en el conjunto de prueba (test set, 23,878 muestras).

Métrica	RNA (MLP)	Random For- est	Diferencia
Accuracy	86.34%	86.57%	RF +0.23%
F1-Score (clase 1)	0.8081	0.8196	RF +0.0115
F1-Score (ponderado)	0.8621	0.8658	RF +0.0037
ROC-AUC	0.9419	0.9424	RF +0.0005

8.2 Reporte de Clasificación Detallado

Table 13: Reporte completo de clasificación — Red Neuronal (RNA).

Clase	Precision	Recall	F1-Score	Support
No Cancelada (0)	0.87	0.91	0.89	15,033
Cancelada (1)	0.84	0.78	0.81	8,845
Accuracy			0.86	23,878
Macro avg	0.86	0.85	0.85	23,878
Weighted avg	0.86	0.86	0.86	23,878

Table 14: Reporte completo de clasificación — Random Forest.

Clase	Precision	Recall	F1-Score	Support
No Cancelada (0)	0.90	0.89	0.89	15,033
Cancelada (1)	0.82	0.82	0.82	8,845
Accuracy			0.87	23,878
Macro avg	0.86	0.86	0.86	23,878
Weighted avg	0.87	0.87	0.87	23,878

8.3 Matrices de Confusión Estimadas

A partir de los reportes de clasificación, se pueden estimar los valores de la matriz de confusión para cada modelo en el conjunto de prueba:

Table 15: Matrices de confusión estimadas en el conjunto de prueba.

Red Neuronal (RNA)				Random Forest	
Real	Pred.	No Canc.	Cancelada	No Canc.	Cancelada
No Cancelada		≈13,680 (91%)	≈1,353 (9%)	≈13,379 (89%)	≈1,654 (11%)
Cancelada		≈1,946 (22%)	≈6,899 (78%)	≈1,592 (18%)	≈7,253 (82%)

8.4 Curvas ROC

Ambos modelos obtuvieron excelentes valores de AUC-ROC, muy superiores al clasificador aleatorio (AUC = 0.50):

Table 16: AUC-ROC comparativo de ambos modelos.

Modelo	AUC-ROC	Interpretación
Red Neuronal (RNA)	0.9419	Discriminación excelente
Random Forest (RF)	0.9424	Discriminación excelente
Clasificador aleatorio	0.5000	Sin capacidad discriminativa

Un $AUC > 0.90$ indica que el modelo tiene una capacidad discriminativa excelente: en el 94% de los casos, el modelo asigna una probabilidad mayor de cancelación a una reserva que realmente se cancelará que a una que no se cancelará, si se elige una de cada clase al azar.

9 Análisis e Interpretación de Resultados

9.1 Interpretación de las Métricas

¿Qué indica un Accuracy del 86% en este contexto? Significa que el modelo predice correctamente el estado de cancelación de 86 de cada 100 reservas. Dado que la clase mayoritaria es “No Cancelada” (63%), un clasificador trivial que predijera siempre “No cancelada” alcanzaría un accuracy del 63%. El modelo mejora este baseline en **23 puntos porcentuales**.

9.2 Interpretación de la Matriz de Confusión

En el contexto hotelero, los errores del modelo tienen implicaciones asimétricas:

Table 17: Implicaciones de los tipos de error en el contexto hotelero.

Tipo de error	Descripción	Impacto en el negocio
Falso Positivo (FP)	Predice cancelación pero la reserva se mantiene	Riesgo de sobrevenir habitaciones; cliente sin cuarto disponible
Falso Negativo (FN)	No predice cancelación pero sí se cancela	Habitación vacía, pérdida directa de ingresos
Más crítico	Falso Negativo	Pérdida de ingresos sin posibilidad de reasignación

Para la industria hotelera, **minimizar los Falsos Negativos (FN) es más importante**, por lo que el *Recall* sobre la clase “Cancelada” es la métrica operativamente más relevante. La RNA obtuvo un Recall de 0.78 para cancelaciones, y el RF un Recall de 0.82.

9.3 Comparación entre Modelos

Table 18: Comparación cualitativa entre RNA y Random Forest.

Aspecto	Red Neuronal (RNA)	Random Forest
Capacidad no lineal	Alta	Media-Alta
Interpretabilidad	Baja (caja negra)	Media (feature importance)
Tiempo de entrenamiento	Mayor	Menor
Robustez a outliers	Media	Alta
Requiere escalado	Sí	No
Manejo de overfitting	Dropout, EarlyStopping, BN	max_depth, min_samples
Recall clase cancelada	0.78	0.82
Precisión clase cancelada	0.84	0.82
AUC-ROC	0.9419	0.9424

¿Cuál modelo es mejor? Las diferencias en ambos modelos son mínimas ($<0.5\%$ en todas las métricas globales). El **Random Forest** resulta marginalmente superior en métricas globales y en Recall de cancelaciones, lo que lo hace más recomendable para la operación hotelera donde minimizar FN es prioritario. Sin embargo, la **RNA** ofrece mayor flexibilidad para escalar a arquitecturas más complejas y podría superar al RF con optimización adicional de hiperparámetros (Bayesian Optimization, arquitecturas más profundas).

9.4 Curva de Aprendizaje

La curva de entrenamiento mostró un comportamiento saludable:

- **Época 1:** Accuracy = 76.01%, val_accuracy = 80.34%.
- **Época 10:** Accuracy = 84.24%, val_accuracy = 85.35%.
- **Época 50 (final):** Accuracy $\approx$ 85.9%, val_accuracy = 87.02%.

La brecha moderada entre train y validación indica que la regularización (Dropout + BatchNorm) controló efectivamente el sobreajuste.

10 Conclusiones

10.1 Aprendizajes

1. **El preprocesamiento es la etapa más crítica:** La eliminación de columnas con fuga de datos (`reservation_status`), el tratamiento correcto de valores nulos y el escalado riguroso (ajustado solo en train) fueron determinantes para el rendimiento real del modelo.
2. **Las Redes Neuronales tienen alta capacidad predictiva en problemas tabulares:** La arquitectura MLP con tres capas ocultas (128→64→32) logró capturar relaciones no lineales complejas, alcanzando métricas superiores al 86% de accuracy y 0.94 de AUC-ROC.
3. **Variables más influyentes:** El tiempo de anticipación de la reserva (`lead_time`), el número de peticiones especiales (`total_of_special_requests`) y el tipo de depósito (`deposit_type`) son los predictores más relevantes, coherentes con el comportamiento real de los clientes hoteleros.
4. **La regularización es fundamental:** La combinación de Dropout, BatchNormalization y EarlyStopping evitó el sobreajuste manteniendo un balance saludable entre sesgo y varianza.
5. **Los modelos ensemble compiten efectivamente:** Random Forest demostró que, en datos tabulares de tamaño moderado, los métodos de ensemble son altamente competitivos con arquitecturas de deep learning, con la ventaja adicional de mayor interpretabilidad.

10.2 Limitaciones Encontradas

- **Desbalance de clases moderado** (63:37) que puede sesgar el modelo hacia la clase mayoritaria. Se podría aplicar SMOTE o ajuste de pesos de clase en la RNA.
- **Alta cardinalidad de country:** El Frequency Encoding puede no capturar toda la información geográfica relevante.
- **Baja interpretabilidad de la RNA:** Dificulta justificar decisiones ante el departamento comercial del hotel.
- **Variables externas no consideradas:** Eventos especiales, temporadas festivas, condiciones económicas o restricciones sanitarias (como la pandemia COVID-19 para datos recientes) podrían mejorar significativamente las predicciones.
- **Datos estáticos:** El modelo no actualiza sus predicciones en tiempo real conforme cambia el comportamiento del mercado.

10.3 Posibles Mejoras

- Aplicar **técnicas de balanceo de clases** (SMOTE, oversampling, `class_weight` en la RNA) para mejorar el Recall en cancelaciones.
- Explorar **modelos de Gradient Boosting** (XGBoost, LightGBM) como comparación adicional, ya que suelen superar a ambos modelos en datos tabulares.
- Implementar **SHAP values** para explicabilidad del modelo RNA.
- Aplicar **Bayesian Optimization** o **Keras Tuner** para optimizar hiperparámetros de la RNA.
- Incorporar **variables temporales externas** (días festivos, eventos en el destino) como features adicionales.
- Implementar el modelo en un sistema de **scoring en tiempo real** que actualice las predicciones conforme se modifica la reserva.

11 Referencias Bibliográficas

Libros:

1. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
<https://www.deeplearningbook.org>
2. Haykin, S. (2009). *Neural networks and learning machines* (3ra ed.). Pearson Education.
3. Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
4. Géron, A. (2022). *Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow* (3ra ed.). O'Reilly Media.

Artículos científicos:

5. Antonio, N., Almeida, A., & Nunes, L. (2019). Hotel booking demand datasets. *Data in Brief*, 22, 41–49. <https://doi.org/10.1016/j.dib.2018.11.126>
6. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1), 1929–1958.

Recursos en línea, dataset:

9. Mock, T., & Bichat, A. (2020). *Hotel bookings* [Dataset]. TidyTuesday. <https://github.com/rfordatas>